



Copyrighted material

This is an integral part of course **Next level Rust** and cannot be used in any context without explicit permission from **CLEUTON SAMPAIO DE MELO JUNIOR** (a.k.a. "the Author"). You can't copy, change, publish, or share with others.

What you need to "unlearn"

What DOESN'T Work in Rust

Transitioning to Rust requires unlearning several patterns that work well in other languages but are either impossible or counterproductive in Rust. The language's design discourages many habits familiar to object-oriented or dynamically typed environments. Knowing what doesn't work in Rust is just as important as understanding what does, because it helps you stop translating and start thinking in Rust.

Forced Pattern Matching: Learning to Embrace `match` Instead of `if/else` Chains

Rust encourages pattern matching over complex conditionals. You can write `if/else` chains, but enums and exhaustive matching are designed to express intent clearly and safely.

```
enum HttpStatus {  
    Ok,  
    NotFound,  
    ServerError,  
    Redirect,  
}  
  
// What not to do - awkward and error-prone  
fn handle_status_bad(status: HttpStatus) -> &'static str {  
    use HttpStatus::*;  
    if let Ok = status {
```

```

        "Success"
    } else if let NotFound = status {
        "Not found"
    } else if let ServerError = status {
        "Server error"
    } else {
        "Redirect"
    }
}

// Preferred Rust style
fn handle_status(status: HttpStatus) -> &'static str {
    match status {
        HttpStatus::Ok => "Success",
        HttpStatus::NotFound => "Not found",
        HttpStatus::ServerError => "Server error",
        HttpStatus::Redirect => "Redirect",
    }
}

```

Pattern matching is more than a cleaner switch. It allows you to destructure complex values and forces you to consider all cases:

```

enum ApiResponse {
    Success { data: String, code: u16 },
    Error { message: String, code: u16 },
    Loading,
}

fn process_response(resp: ApiResponse) -> String {
    match resp {
        ApiResponse::Success { data, code } => format!("Data: {}, Code: {}", data, code),
        ApiResponse::Error { message, code } => format!("Error {}: {}", code, message),
        ApiResponse::Loading => "Loading...".into(),
    }
}

```

This cannot be done safely or cleanly with `if/else`. Match is how Rust encourages clear, exhaustive, and expressive control flow.

Controlled Mutability: When and How to Use `mut` Idiomatically

In Rust, mutability is opt-in and tightly controlled. You must declare it explicitly and respect borrowing rules. Overusing `mut` or fighting the borrow checker usually signals design problems.

```

// Invalid: trying to mutate an immutable variable
fn invalid_mutation() {

```

```

    let x = 5;
    // x = 6; // Compile error
}

// Valid: declared as mutable
fn valid_mutation() {
    let mut x = 5;
    x += 1;
    println!("x = {}", x);
}

```

Overusing mutability makes your code harder to reason about:

```

fn excessive_mutation() {
    let mut numbers = vec![1, 2, 3, 4, 5];
    for i in 0..numbers.len() {
        numbers[i] *= 2;
    }
    for i in 0..numbers.len() {
        numbers[i] -= 1;
    }
    numbers.sort();
    println!("{:?}", numbers);
}

```

Better: declarative and localized mutation:

```

fn better_mutation() {
    let numbers = vec![1, 2, 3, 4, 5];
    let processed: Vec<_> = numbers
        .into_iter()
        .map(|x| x * 2)
        .map(|x| x - 1)
        .sorted() // from itertools::Itertools
        .collect();
    println!("{:?}", processed);
}

```

Borrowing is also enforced strictly:

```

// Won't compile: mixed immutable and mutable borrows
fn bad_borrowing() {
    let mut names = vec!["Alice".to_string()];
    let first = &names[0];
    // names.push("Bob".to_string()); // Error
}

```

```

    // println!("{}", first); // Also error
}

// Fixed: use references in the right order
fn good_borrowing() {
    let mut names = vec!["Alice".to_string()];
    println!("First: {}", names[0]);
    names.push("Bob".to_string());
}

```

Using `Rc<RefCell<T>>` everywhere to “get Java-style shared mutability” is a code smell in Rust. It hides aliasing rules, defers errors to runtime (borrow panics), and makes reasoning about ownership harder. Prefer designs where mutation happens through unique `&mut` borrows, or push shared changes through clear boundaries (message passing, explicit APIs). Use interior mutability sparingly and locally.

Trying to mimic Java-style shared mutability is a “bad smell” in Rust:

Bad (Java-ish, “shared mutable object graph”):

```

use std::rc::Rc;
use std::cell::RefCell;

#[derive(Debug)]
struct Node {
    name: String,
    children: Vec<Rc<RefCell<Node>>>,
}

impl Node {
    fn add_child(self_rc: &Rc<RefCell<Node>>, child: Rc<RefCell<Node>>) {
        // Hidden runtime cost + possible borrow panics
        self_rc.borrow_mut().children.push(child);
    }

    fn rename_and_touch_all(self_rc: &Rc<RefCell<Node>>, new_name: &str) {
        let mut me = self_rc.borrow_mut();
        me.name = new_name.to_string();

        // Oops: still holding a mutable borrow, then we re-enter borrow on
        // a child.
        // In non-trivial code this easily becomes a runtime panic (already
        // borrowed).
        for ch in &me.children {
            ch.borrow_mut().name.push('!'); // may panic at runtime
        }
    }
}

```

Problems: no clear owner, mutation scattered, borrow rules enforced at runtime, easy to panic, hard to test and reason about.

Good 1 (owning tree; mutate with `&mut` and stable IDs):

```
#[derive(Debug)]
struct Node {
    name: String,
    children: Vec<usize>, // indices into Arena.nodes
}

struct Arena {
    nodes: Vec<Node>,
}

impl Arena {
    fn new_root(name: &str) -> Self {
        Self { nodes: vec![Node { name: name.into(), children: vec![] }] }
    }
    fn add_child(&mut self, parent_idx: usize, name: &str) -> usize {
        let child_idx = self.nodes.len();
        self.nodes.push(Node { name: name.into(), children: vec![] });
        self.nodes[parent_idx].children.push(child_idx);
        child_idx
    }
    fn rename(&mut self, idx: usize, to: &str) {
        self.nodes[idx].name = to.into();
    }
    fn touch_children(&mut self, idx: usize) {
        // Single &mut self governs all mutation; no runtime borrow checks.
        let children = self.nodes[idx].children.clone();
        for c in children { self.nodes[c].name.push('!'); }
    }
}
```

Benefits: one clear owner (`Arena`), mutations via `&mut` (compile-time enforced), no borrow panics, cheap stable references via indices, easy to reason about.

An **arena** is just a container that owns many objects in one place, usually a `Vec`. Instead of giving out mutable references everywhere, you give out **indices** (like IDs) into the arena. The arena stays the single owner, so you can safely mutate objects through `&mut self` without `Rc<RefCell>` tricks.

In the example, `Arena` holds a `Vec<Node>`. Each `Node` stores the indices of its children. To add a child, you push a new `Node` into the vector and record its index. To mutate, you call methods on the arena (like `rename` or `touch_children`), which borrow the arena mutably once and then update as needed.

This way all ownership is clear, mutation is explicit and compile-time safe, and you avoid runtime borrow errors from `RefCell`.

Rule of thumb: reach for `Rc<RefCell<T>>` only at narrow boundaries where you truly cannot obtain `&mut` (e.g., GUI callbacks, self-referential registration). For general models and data structures, prefer a single owner with `&mut` access, arenas with indices, or explicit “replace with a new value” strategies.

Rust's mutability model leads to safer and more predictable code. If something is shared, it should not be mutated. If something is mutated, it should not be shared.

Error Handling Without Exceptions: Transforming Control Flow Into Types

Rust avoids exceptions entirely. It uses types like `Result` and `Option` to make error conditions explicit and enforce proper handling at compile time.

```
#[derive(Debug)]
enum MathError {
    DivisionByZero,
    Overflow,
}

fn safe_divide(a: i32, b: i32) -> Result<i32, MathError> {
    if b == 0 {
        Err(MathError::DivisionByZero)
    } else {
        a.checked_div(b).ok_or(MathError::Overflow)
    }
}
```

Ignoring errors leads to panics:

```
fn bad_handling() {
    let result = safe_divide(10, 0);
    // result.unwrap(); // Panic
}
```

Instead, propagate errors properly:

```
fn better_handling(x: i32, y: i32) -> Result<i32, MathError> {
    let div = safe_divide(x, y)?;
    Ok(div + 1)
}
```

Handling multiple error types is also structured:

```
#[derive(Debug)]
enum UserError {
    NotFound,
}

#[derive(Debug)]
enum AppError {
    Math(MathError),
}
```

```

    User(UserError),
}

impl From<MathError> for AppError {
    fn from(e: MathError) -> Self {
        AppError::Math(e)
    }
}

impl From<UserError> for AppError {
    fn from(e: UserError) -> Self {
        AppError::User(e)
    }
}

struct User {
    name: String,
    id: u32,
}

fn find_user(id: u32) -> Result<User, UserError> {
    if id == 42 {
        Ok(User { name: "Alice".into(), id })
    } else {
        Err(UserError::NotFound)
    }
}

fn full_operation() -> Result<i32, AppError> {
    let user = find_user(42)?;
    let result = safe_divide(100, 10)?;
    Ok(result + user.id as i32)
}

```

Optional values are handled with `Option`:

```

fn find_user_by_email(email: &str) -> Option<User> {
    if email == "valid@example.com" {
        Some(User { name: "Alice".into(), id: 1 })
    } else {
        None
    }
}

fn example_usage() -> Result<(), String> {
    let user = find_user_by_email("invalid@example.com").ok_or("Not found")?;
    println!("Found {}", user.name);
    Ok(())
}

```

Rust makes error handling explicit and type-safe. You know when something might fail, and you must decide what to do about it.

Final Notes

Every example in this chapter reflects a mindset shift. Rust does not allow you to ignore edge cases, or to delay error discovery to runtime. It rewards precision and discourages guesswork. The compiler will stop you from writing unsafe or incorrect code. Learning to trust it and work with its rules is what makes Rust development uniquely powerful.